

Frontier Intelligence Pipeline — Architecture & Production Design

Export this file to `architecture.pdf` for submission (Print → Save as PDF, or `pandoc docs/architecture.md -o architecture.pdf`). Kept ≤3 pages as required.

0. System overview

A streaming producer/consumer pipeline. **Scrapers** are thin async generators that yield raw dicts carrying provenance (`source_url`). Each candidate passes through normalization, provenance checks, freshness filtering where applicable, deduplication, optional LLM structuring, entity resolution, schema validation, and output emission.

The implementation keeps records streaming rather than materializing the full corpus. Shared concurrency and rate-limiting controls prevent a fast source from overwhelming downstream stages.

Validated implementation stack: `asyncio` + `httpx` for I/O; `aiolimiter` for rate limiting; `Pydantic` for schema contracts; `rapidfuzz` for entity resolution; `SQLite` for the local dedupe/freshness ledger; `CSV` for exported tabular output; `Gemini/Groq/DeepSeek` for optional LLM enrichment. The production-scale design described below extends these components with shared infrastructure such as `Redis`, `PostgreSQL`, object storage, and a distributed work queue.

1. Scale Strategy — collecting 500,000 records without manual intervention

Principle: volume is an infrastructure parameter, not a code rewrite.

- **Source-appropriate acquisition.** Prefer official/structured endpoints (arXiv Atom API, Algolia-backed directories like YC, job-board JSON APIs, RSS) over expensive HTML scraping. These paginate cheaply and can scale from thousands to hundreds of thousands of records through existing pagination loops.
- **Horizontal fan-out.** The production unit of work is a URL/record. Stateless crawler workers can pull work from a shared queue such as `Redis Streams` or `SQS` while sharing a distributed dedupe ledger. Throughput increases by adding workers rather than changing business logic.
- **Bounded politeness.** Global rate limits and per-host concurrency controls allow parallel collection across domains without concentrating excessive load on a single host.
- **Backpressure.** Bounded producer/consumer queues prevent fast ingestion stages from overwhelming slower processing stages.
- **Checkpointing.** The dedupe ledger records claimed identities so interrupted runs can resume without repeatedly processing the same records.

The current implementation validates these behaviors on a single node with `SQLite`; `Redis`-backed coordination is the production-scale extension.

2. Handling 413s & 429s across concurrent extractions

413 — *payload too large*

Before every LLM call, the orchestrator calculates a provider-specific source text budget and uses token counting plus salient truncation when necessary. This minimizes context-overflow failures before the provider request.

If a provider nevertheless returns 413, the orchestrator reduces the payload budget and retries the **same provider** with the smaller payload. Repeated 413 responses continue reducing the budget until the configured minimum threshold is reached; at that point the provider is abandoned and the fallback chain continues.

429 — *rate limiting*

The LLM orchestrator retries the **same provider** with bounded exponential full-jitter backoff. Provider-supplied `Retry-After` values are respected when reasonable; excessively long retry delays cause the provider to be temporarily marked unavailable rather than blocking the entire pipeline.

After the configured 429 retry budget is exhausted, a provider circuit is opened temporarily. Subsequent records skip that provider while its circuit is open and continue through the remaining fallback chain.

The fallback order is:

Gemini → **Groq** → **DeepSeek**

This prevents a single provider quota from stalling the complete extraction pipeline.

If every configured provider fails, the record is quarantined and counted.

No guessed or fabricated extraction is emitted.

3. Freshness & deduplication

- **Content-addressed identity.** Record identity is derived from the normalized canonical source URL. The local SQLite ledger provides atomic single-node claiming semantics. A production deployment can replace this with Redis SET `NX` or an equivalent distributed atomic claim.
 - **Freshness window.** Publication dates are normalized to timezone-aware UTC. Records from freshness-sensitive verticals are accepted only when they fall within the configured `FRESHNESS_WINDOW_HOURS` window.
 - **Missing-date heuristic.** When a reliable publication date is unavailable, the ledger's prior-seen state is used to distinguish newly discovered records from records already processed.
 - **Failure recovery.** Once an identity is claimed, downstream failures such as LLM failure release the claim so the record remains retryable rather than becoming a permanent duplicate.
- This makes transient provider/network failures recoverable without weakening deduplication guarantees.

4. Storage strategy

Current validated implementation

The trial implementation uses:

- **SQLite** for the dedupe/freshness ledger.
- **CSV** files for the five output verticals and entity-mapping log.
- **Pydantic** models for validation and stable record contracts.

This lightweight arrangement keeps the local trial reproducible and easy to inspect.

Production-scale extension

For a distributed deployment:

- **PostgreSQL** can become the primary durable record store, using JSONB, indexes, ACID transactions, and upserts.
- **Redis** can provide distributed dedupe claims and hot coordination.
- **Neo4j**, or PostgreSQL plus graph-oriented structures, can represent relationships such as startups → founders → products → papers → repositories → jobs.
- **pgvector** can complement deterministic entity resolution with semantic similarity.
- **S3/GCS** can retain raw source snapshots for auditability and re-processing.

These components are production-design targets rather than requirements of the local trial run.

5. Anti-bot strategy

The acquisition architecture uses tiered escalation, from cheapest to most expensive:

- Direct HTTP/API requests for structured sources.
- Realistic headers and cookie/session handling where appropriate.
- Browser automation for sources that genuinely require JavaScript rendering.
- Production deployments may add approved proxy infrastructure where

permitted by the target site's terms.

The system should degrade politely rather than aggressively attempting to defeat CAPTCHAs or access controls. The objective is durable data acquisition, not circumvention.

Browser/proxy infrastructure is an escalation path in the production design; the core validated trial path remains focused on structured and HTTP-based sources.

6. Data-integrity guarantees

- `source.url` is supplied by the ingestion layer and is never generated by the LLM.

- The extraction system prompt explicitly forbids inference: when a requested field is absent from the supplied text, the model must return `null`.

- LLM output is merged only into permitted content fields; protected provenance and identity fields cannot be overwritten by extraction output.

- Records are validated against the Pydantic schema contract before emission.

- Provider failures are quarantined and counted rather than emitted with fabricated values.

- Dedupe claims are released after downstream processing failures so transient failures remain retryable.

- The production design supports retaining raw source material for audit and re-processing.

The result is a failure-tolerant pipeline in which unavailable providers, rate-limit events, oversized payloads, and transient processing errors degrade availability without silently corrupting the dataset.